Supplementary Material: Pedagogical State Tracking: Extending Knowledge Tracing to Whole-Classroom Discourse

This Supplementary Material covers every model from first principles: motivation, mathematical formulation, prior choices, forward algorithm derivation, worked numerical examples, and code. Updated version of this Supplementary Material can be found on https://github.com/drake1729/AIML_System

June 29, 2026

Contents

1 What Are We Modelling?

1.1 The Problem

Teaching is sequential: a teacher speaks, students respond, the cognitive climate of the room shifts. The paper asks: *can we track whether a classroom is in a "productive discourse regime", one that activates 21st-Century Skills (21CS), utterance by utterance?*

This is discourse Pedagogical State Tracking. It adapts Bayesian Knowledge Tracing (BKT), originally designed for tracking a single student's mastery of a single topic, to the whole-classroom level.

1.2 Data at a Glance

Property	**Value**	**Note**
Episodes E	49 (48 usable)	31 Science, 18 Math; Grades 6–12
Total lines	19,181	Median episode: 384 lines
Mean duration	28.7 min	SD 4.8; range 12–39 min
Source	Indian classroom (48/49)	1 episode from vignette named, 'Volcanoes? Here?'
Feature	**Dimensions**	**Description**
EP (Educational Practice)	3	Behaviourism, Cognitivism, Constructivism
KD (Knowledge Dimension)	4	Factual, Conceptual, Procedural, Metacognitive
CP (Cognitive Process)	6	Remembering, Understanding, Applying, Analysing, Evaluating, Creating
Outcome		
$O_t \in \{0, 1\}$	1	1 if any 21CS active at line t (prevalence 0.482)
21CS	4	Problem Solving, Creativity, Collaborative Learning, Critical Thinking

1.3 The Covariate Vector

At every line t, we have a 14-dimensional binary vector (13 content features + 1 intercept):

$$\mathbf{z}_t = \big[\underbrace{\text{Beh, Cog, Con}}_{\text{3 EP}} \,\|\, \underbrace{\text{Fac, Con, Pro, Met}}_{\text{4 KD}} \,\|\, \underbrace{\text{Rem, Und, App, Ana, Eva, Cre}}_{\text{6 CP}} \,\|\, \underbrace{1}_{1}\big] \in \{0,1\}^{13}\times\{1\} \tag{1}$$

1.4 The Three Models and Their Trade-Offs

Model	Core Assumption	Temporal?	Content-Driven?	AUC
BLFM	Episode = bag of lines	**No**	Yes (mean)	0.772
PST	HMM, fixed transitions	**Yes**	**No**	0.928*
IO-BKT	HMM, content-driven trans.	**Yes**	**Yes**	**0.873**

* Inflated by autocorrelation, explained in §3.9.

The models are not competitors: they *answer different questions* and form a deliberate complexity ladder. We start with the simplest.

2 Model 1: BLFM — Bayesian Latent Factor Model

2.1 Core Question

BLFM Answers

Does the average mix of pedagogical moves in an episode predict how often 21CS will be active, regardless of the order in which those moves appear?

2.2 The Exchangeability Assumption

BLFM makes one fundamental assumption: lines within an episode are **exchangeable** — like balls drawn from a bag. Shuffling the lines in any order produces the same prediction.

This is, at first, startling. Why would we ignore temporal order?

Why Exchangeability Is a Reasonable Starting Point

PST (Model 2) estimates $\hat{p}_L = 0.073$: the classroom changes regime roughly *once every 14 lines*. An episode with 384 lines sees only $\sim$27 transitions. The classroom spends most of its time locked in one regime. Therefore, the *fraction* of lines spent in each regime (captured by the mean feature vector) is a reasonable proxy for the dominant regime quality. Order only matters at the few transition moments.

2.3 The Sufficient Statistic: Episode-Level Aggregation

For episode e with T_e lines, BLFM collapses the full sequence into two summaries:

$$\bar{\mathbf{x}}_e = \frac{1}{T_e}\sum_{t=1}^{T_e} \mathbf{z}_t \in [0,1]^{13} \qquad \text{(mean feature vector)} \tag{2}$$

$$\bar{y}_e = \frac{1}{T_e}\sum_{t=1}^{T_e} O_t \in (0,1) \qquad \text{(observed 21CS rate)} \tag{3}$$

The 19,181 individual lines collapse to **48 episode-level observations**. BLFM regresses $\bar{y}_e$ on $\bar{\mathbf{x}}_e$.

Computing the Sufficient Statistic

Suppose Episode 7 has $T_e = 100$ lines:

- 70 lines: Behaviourism=1 $\Rightarrow$ $\bar{x}_{\text{Beh}} = 0.70$
- 20 lines: Cognitivism=1 $\Rightarrow$ $\bar{x}_{\text{Cog}} = 0.20$
- 10 lines: Constructivism=1 $\Rightarrow$ $\bar{x}_{\text{Con}} = 0.10$
- 40 lines: $O_t = 1$ $\Rightarrow$ $\bar{y}_7 = 0.40$

BLFM sees $(\bar{\mathbf{x}}_7, 0.40)$ as a **single data point**. It does not know which specific lines had 21CS active, nor whether the constructivist moves came at the beginning or end.

2.4 Model Specification

2.4.1 Step 1 — Automatic Relevance Determination (ARD) Prior

Instead of assigning all features the same prior variance, BLFM gives each feature f its own *precision* α_f (inverse variance):

$$\alpha_f \sim \text{Gamma}(1,\, 1) \qquad \forall\, f = 1, \ldots, P \tag{4}$$

$$w_f \sim \mathcal{N}\left(0,\, \frac{1}{\alpha_f}\right) \qquad \forall\, f \tag{5}$$

ARD Intuition — Automatic Feature Selection

Think of α_f as a "tightness dial" for feature f.

- **Irrelevant feature**: The data provides no signal about w_f. α_f stays large (the Gamma$(1, 1)$ prior has mode 0, meaning it prefers small but the posterior is driven by likelihood). Large $\alpha_f \Rightarrow$ small variance $1/\alpha_f \Rightarrow w_f \approx 0$. The feature is effectively *switched off*.
- **Relevant feature**: The data pulls α_f toward small values, widening the prior and allowing w_f to be non-zero.

Result: you never need to manually select which of the 13 EP/KD/CP features to include. The model selects them automatically.

Implementation note. The standard deviation of w_f is $1/\sqrt{\alpha_f}$, so in NumPyro: `dist.Normal(0, 1.0/jnp.sqrt(alpha))`. The precision is α_f, not the standard deviation.

2.4.2 Step 2 — Intercept and Random Effects

$$b \sim \mathcal{N}(0,\, 1) \tag{6}$$

$$\sigma_{\text{cell}} \sim \text{Half-Normal}(1) \tag{7}$$

$$u_e \mid \sigma_{\text{cell}} \sim \mathcal{N}(0,\, \sigma^2_{\text{cell}}) \tag{8}$$

u_e is a **per-episode random effect** capturing everything the content features cannot explain: the specific teacher's charisma, the class's prior engagement, time of day, and so on. Episodes sharing the same Subject×Grade cell share a common σ_{cell}, creating **partial pooling**: a Grade-9 Science episode borrows strength from other Grade-9 Science episodes when estimating σ_{cell}.

2.4.3 Step 3 — Predicted Episode Rate

$$\boxed{\mu_e = \sigma\left(\mathbf{w}^\top \bar{\mathbf{x}}_e + u_e + b\right)} \qquad \mu_e \in (0, 1) \tag{9}$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid/logistic function that maps any real number to $(0, 1)$.

2.4.4 Step 4 — Beta Regression Likelihood

Since $\bar{y}_e$ is a proportion in $(0, 1)$, a Beta likelihood is the natural choice:

$$\phi \sim \text{Gamma}(5,\ 0.1) \quad \text{(concentration/precision)} \tag{10}$$

$$\bar{y}_e \sim \text{Beta}(\mu_e\phi,\ \ (1-\mu_e)\phi) \tag{11}$$

Beta Distribution Mean and Variance

If $Y \sim \text{Beta}(a, b)$, then:

$$\mathbb{E}[Y] = \frac{a}{a+b}, \qquad \text{Var}[Y] = \frac{\mathbb{E}[Y]\big(1-\mathbb{E}[Y]\big)}{\phi+1}$$

In our parameterisation: $a = \mu_e\phi$, $b = (1-\mu_e)\phi$, so:

$$\mathbb{E}[\bar{y}_e] = \frac{\mu_e\phi}{\mu_e\phi + (1-\mu_e)\phi} = \mu_e$$

The Beta likelihood says: "the observed 21CS rate $\bar{y}_e$ should be close to the predicted rate μ_e, with concentration controlled by ϕ." Larger ϕ = tighter fit. The prior $\phi \sim \text{Gamma}(5, 0.1)$ has mean 50, encoding moderate concentration.

2.5 Full Generative Story

BLFM Generative Process (for all episodes simultaneously)

1. For each feature f: draw precision $\alpha_f \sim \text{Gamma}(1, 1)$
2. For each feature f: draw weight $w_f \sim \mathcal{N}(0, 1/\alpha_f)$
3. Draw global intercept $b \sim \mathcal{N}(0, 1)$
4. Draw cell variance $\sigma_{\text{cell}} \sim \text{Half-Normal}(1)$
5. For each episode e: draw episode effect $u_e \sim \mathcal{N}(0, \sigma^2_{\text{cell}})$
6. For each episode e: compute $\mu_e = \sigma(\mathbf{w}^\top \bar{\mathbf{x}}_e + u_e + b)$
7. Draw concentration $\phi \sim \text{Gamma}(5, 0.1)$
8. For each episode e: **observe** $\bar{y}_e \sim \text{Beta}(\mu_e\phi,\ (1-\mu_e)\phi)$

2.6 Worked Numerical Example

Suppose MCMC returns these posterior means: $w_{\text{Beh}} = -1.5$, $w_{\text{Cog}} = -0.4$, $w_{\text{Con}} = +1.8$, $w_{\text{App}} = +2.1$ (all other $w_f \approx 0$), $b = -0.5$.

Episode A (mostly Behaviourist, low cognitive demand):

$$\bar{x}_{\text{Beh}} = 0.80, \quad \bar{x}_{\text{App}} = 0.05, \quad u_A = +0.1$$

$$\text{logit}(\mu_A) = (-1.5)(0.80) + (2.1)(0.05) - 0.5 + 0.1 = -2.0 \ \Rightarrow\ \hat{\mu}_A = \sigma(-2.0) = \mathbf{0.12}$$

Episode B (more Constructivist, higher Applying):

$$\bar{x}_{\text{Con}} = 0.30, \quad \bar{x}_{\text{App}} = 0.25, \quad u_B = -0.1$$

$$\text{logit}(\mu_B) = (1.8)(0.30) + (2.1)(0.25) - 0.5 - 0.1 = +0.465 \;\Rightarrow\; \hat{\mu}_B = \sigma(0.465) = \mathbf{0.61}$$

The model correctly assigns a much higher predicted 21CS rate to Episode B.

2.7 Code

```
def blfm_model(x_bar, y_bar, cell_idx, n_cells, P):
    """
    x_bar    : shape (E, 13)  -- mean feature vector per
       episode
    y_bar    : shape (E,)     -- observed 21CS rate per episode
    cell_idx : shape (E,)     -- Subject x Grade cell index
    n_cells  : int            -- number of unique cells
    P        : int            -- number of features (13)
    """
# Step 1: ARD priors
    # One precision alpha_f per feature
    alpha = numpyro.sample("alpha",
                           dist.Gamma(1.0, 1.0).expand([P]))
    # Weight: w_f ~ N(0, 1/alpha_f)
    # std = 1/sqrt(alpha) because Var(w_f) = 1/alpha_f
    w = numpyro.sample("w",
                       dist.Normal(jnp.zeros(P),
                                   1.0 / jnp.sqrt(alpha)))
    # Step 2: Global intercept
    b = numpyro.sample("b", dist.Normal(0.0, 1.0))

    # Step 3: Subject x Grade random effects
    # sigma_cell controls how much episodes within a cell vary
    sigma_cell = numpyro.sample("sigma_cell",
                                dist.HalfNormal(1.0))
    # u_cell[c] ~ N(0, sigma_cell^2) for each cell c
    u_cell = numpyro.sample("u_cell",
                 dist.Normal(jnp.zeros(n_cells),
                             sigma_cell * jnp.ones(n_cells)))
    # Map each episode to its cell's random effect
    u_e = u_cell[cell_idx]              # shape: (E,)

    # Step 4: Predicted episode 21CS rate
    # logit(mu_e) = w^T x_bar_e + u_e + b
    logit_mu = jnp.dot(x_bar, w) + u_e + b
    mu = jax.nn.sigmoid(logit_mu)       # mu_e in (0, 1)

    # Step 5: Beta precision
    # Gamma(5, 0.1): mean=50, represents moderate concentration
    phi = numpyro.sample("phi", dist.Gamma(5.0, 0.1))
```

```
40
41      # Step 6: Beta likelihood
42      a  = mu * phi + 1e-6                # alpha shape param
43      b_ = (1.0 - mu) * phi + 1e-6       # beta shape param
44      # Clamp y_bar to open interval: Beta requires (0,1)
           strictly
45      y_bar_clamped = jnp.clip(y_bar, 1e-4, 1.0 - 1e-4)
46      numpyro.sample("y_bar", dist.Beta(a, b_), obs=y_bar_clamped
           )
```

Listing 1: BLFM: Full model definition in NumPyro

2.7.1 Key Implementation Details Explained

`1.0/jnp.sqrt(alpha)`
: The standard deviation of w_f is $\alpha_f^{-1/2}$, because $\mathrm{Var}(w_f) = 1/\alpha_f$. Always distinguish precision (α) from standard deviation.

`u_cell[cell_idx]`
: `cell_idx[i]` stores which Subject×Grade cell episode i belongs to. This single indexing operation maps each of the E episodes to its cell's random effect.

`jnp.clip(y_bar, 1e-4, 1-1e-4)`
: The Beta distribution requires strict open-interval support. Any episode with $\bar{y}_e = 0$ (no 21CS at all) or $\bar{y}_e = 1$ would produce $\log \mathrm{Beta}(-\infty)$. The clamp prevents this.

`1e-6` **stabiliser on** a **and** b_-
: When $\mu \approx 0$ or $\mu \approx 1$, one of the Beta shape parameters would be nearly zero, causing numerical underflow. The small additive constant stabilises the computation.

2.8 Running MCMC

```
1 kernel_blfm = NUTS(blfm_model, target_accept_prob=0.90)
2 mcmc_blfm   = MCMC(kernel_blfm,
3                    num_warmup=500,     # adapt step size for 500
                        steps
4                    num_samples=1000,   # draw 1000 posterior
                        samples
5                    num_chains=2,       # run 2 independent
                        chains
6                    progress_bar=True)
7 mcmc_blfm.run(jax.random.PRNGKey(42),
8               x_bar_jax, y_bar_jax,
9               cell_idx_jax, n_cells, P)
```

Listing 2: BLFM: NUTS sampler configuration

NUTS (No-U-Turn Sampler) automatically tunes step size and trajectory length during warmup, making it far more efficient than basic Metropolis-Hastings for high-dimensional

posteriors. `target_accept_prob=0.90` means: aim for 90% acceptance, which NUTS achieves by tuning step sizes during warmup.

2.9 What BLFM Cannot Answer

BLFM Blind Spots

- What is the probability the classroom is productive *right now*, at line 47?
- Did the Constructivist move at line 32 *cause* a regime shift?
- How does the productive state evolve *across* the lesson arc?

These require a sequential model with a latent state. Enter PST.

3 Model 2: PST — Probabilistic Skill Tracing (Sequential BKT)

3.1 Core Question

PST Answers

How often does the classroom switch between productive and unproductive discourse? And what is the probability of being in the productive regime at each line, without knowing anything about the pedagogical content?

3.2 Hidden Markov Model (HMM) Primer

PST models the classroom as a two-state Hidden Markov Model. We have:

- A **latent state** $M_t \in \{0, 1\}$ at each line t — never directly observed.
- A **visible observation** $O_t \in \{0, 1\}$ — whether 21CS is active.

The HMM diagram below shows the structure:

3.3 PST Parameters

Symbol	Meaning	Prior
π_0	$P(M_1 = 1)$: probability episode starts in the productive state	$\text{Beta}(2, 3)$
p_L	Learning rate: $P(M_t{=}1 \mid M_{t-1}{=}0)$ per line	$\text{Beta}(1.5, 20)$
p_F	Forgetting rate: $P(M_t{=}0 \mid M_{t-1}{=}1)$ per line	$\text{Beta}(1, 50)$
p_S	Slip: $P(O_t{=}0 \mid M_t{=}1)$	**Fixed** $= 0.015$
p_G	Guess: $P(O_t{=}1 \mid M_t{=}0)$	**Fixed** $= 0.009$

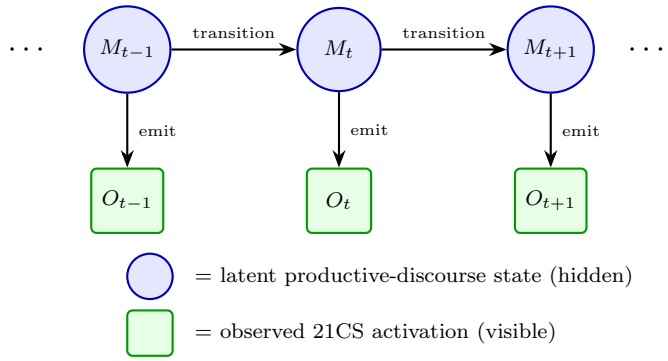


Figure 1: PST graphical model. The discourse state M_t transitions between productive (M=1) and unproductive (M=0) with fixed probabilities p_L (learning) and p_F (forgetting), independent of pedagogical content. The observed 21CS activation O_t is emitted with slip probability p_S and guess probability p_G. No covariate vector $\mathbf{z}_t$ is present: content-blindness is intentional, isolating pure regime dynamics from pedagogical content. Compare IO-BKT (Fig. 2), which adds an orange $\mathbf{z}_t$ row and content-driven transition probabilities.

3.3.1 The Transition Matrix

At every time step, the probability of moving between states is constant (PST ignores content):

$$\mathbf{A} = \begin{pmatrix} P(M_t{=}0 \mid M_{t-1}{=}0) & P(M_t{=}1 \mid M_{t-1}{=}0) \\ P(M_t{=}0 \mid M_{t-1}{=}1) & P(M_t{=}1 \mid M_{t-1}{=}1) \end{pmatrix} = \begin{pmatrix} 1-p_L & p_L \\ p_F & 1-p_F \end{pmatrix} \tag{12}$$

The posterior mean $\hat{p}_L = 0.073$ means each line has only a 7.3% chance of transitioning from unproductive to productive. Regimes are sticky.

3.3.2 Emission Probabilities

$$P(O_t = 1 \mid M_t = 1) = 1 - p_S = 0.985 \quad \text{(productive} \Rightarrow \text{21CS almost always observed)} \tag{13}$$

$$P(O_t = 0 \mid M_t = 1) = p_S = 0.015 \quad \text{(“slip”: productive but no 21CS)} \tag{14}$$

$$P(O_t = 1 \mid M_t = 0) = p_G = 0.009 \quad \text{(“guess”: unproductive but 21CS fires)} \tag{15}$$

$$P(O_t = 0 \mid M_t = 0) = 1 - p_G = 0.991 \quad \text{(unproductive} \Rightarrow \text{no 21CS almost always)} \tag{16}$$

Why Call Them “Slip” and “Guess”?

These terms originate from BKT in tutoring systems (Corbett & Anderson, 1994):

- **Slip** ($p_S = 0.015$): Even when the classroom is genuinely in the productive regime, expert raters occasionally miss 21CS activation (momentary pause, annotation noise). This is rare: 21CS is almost always visible when the regime is truly productive.
- **Guess** ($p_G = 0.009$): Even in an unproductive regime, a single spontaneous student question might briefly activate 21CS. Also very rare.

Low slip and guess values mean the states are *nearly fully separable* by their emissions.

3.3.3 Why Are p_S and p_G Fixed?

This is a critical design decision. If we estimated p_S and p_G freely, we would face **label switching**: mathematically, swapping the labels "productive" and "unproductive" (and simultaneously swapping emission parameters) gives the exact same likelihood. MCMC would explore both interpretations, producing posterior means of $p_S \approx p_G \approx 0.5$ — meaningless.

Label Switching Visualised

Imagine the model has two possible solutions:

	Solution A (correct)	**Solution B (flipped)**
State 1 meaning	Productive	Unproductive
p_S (State 1 slip)	0.015	0.991
p_G (State 0 guess)	0.009	0.985
Likelihood	$\mathcal{L}$	$\mathcal{L}$ (identical!)

Both solutions have the same likelihood. Without content features to distinguish states, MCMC mixes freely between them. Fixing p_S and p_G at values from IO-BKT (which can identify them via content features) resolves the ambiguity.

3.4 The Forward Algorithm: Step-by-Step Derivation

The forward algorithm is the engine of PST. It computes the marginal likelihood $P(O_1, \ldots, O_{T_e} \mid \theta)$ *exactly* in $O(T_e)$ time, by recursively integrating out the hidden states M_t.

3.4.1 Why Not Enumerate All State Sequences?

A brute-force approach would sum over all 2^{T_e} state sequences. For $T_e = 384$, this is $\sim 10^{115}$ terms — impossible. The forward algorithm avoids this by exploiting the **Markov property**: M_t depends only on M_{t-1}, not the full history.

3.4.2 Notation

Define the **filtered joint probability** at time t:

$$\alpha_t^{(m)} \;=\; P(M_t = m,\; O_1, O_2, \ldots, O_t \mid \theta) \tag{17}$$

This is the probability that: (a) we have seen observations $O_1, \ldots, O_t$, *and* (b) the state at time t is m.

3.4.3 Initialisation ($t = 1$)

$$\alpha_1^{(0)} = P(M_1 = 0) \cdot P(O_1 \mid M_1 = 0) = (1 - \pi_0) \cdot e_0(O_1) \tag{18}$$

$$\alpha_1^{(1)} = P(M_1 = 1) \cdot P(O_1 \mid M_1 = 1) = \pi_0 \cdot e_1(O_1) \tag{19}$$

where the emission shorthand is:

$$e_m(O_t) = \begin{cases} 1 - p_G & \text{if } m = 0,\ O_t = 0 \\ p_G & \text{if } m = 0,\ O_t = 1 \\ p_S & \text{if } m = 1,\ O_t = 0 \\ 1 - p_S & \text{if } m = 1,\ O_t = 1 \end{cases} \tag{20}$$

Normalise: $c_1 = \alpha_1^{(0)} + \alpha_1^{(1)} + \varepsilon$; then $\alpha_1^{(m)} \leftarrow \alpha_1^{(m)} / c_1$.

3.4.4 Recursion ($t \geq 2$): Predict–Update

Predict step — marginalise over the previous state:

$$\tilde{\alpha}_t^{(0)} = \alpha_{t-1}^{(0)} (1 - p_L) + \alpha_{t-1}^{(1)} p_F \tag{21}$$

$$\tilde{\alpha}_t^{(1)} = \alpha_{t-1}^{(0)} p_L + \alpha_{t-1}^{(1)} (1 - p_F) \tag{22}$$

Update step — weight by the emission probability:

$$\alpha_t^{(0)} \propto \tilde{\alpha}_t^{(0)} \cdot e_0(O_t) \tag{23}$$

$$\alpha_t^{(1)} \propto \tilde{\alpha}_t^{(1)} \cdot e_1(O_t) \tag{24}$$

Normalise with $c_t = \alpha_t^{(0)} + \alpha_t^{(1)} + \varepsilon$.

3.4.5 Log-Likelihood

$$\log P(O_1, \ldots, O_{T_e} \mid \theta) \ = \ \sum_{t=1}^{T_e} \log c_t \tag{25}$$

This is the quantity NUTS maximises (in expectation) during MCMC.

3.5 Worked Numerical Forward Pass

Parameters: $\pi_0 = 0.4$, $p_L = 0.07$, $p_F = 0.02$, $p_S = 0.015$, $p_G = 0.009$.

Observations: $[O_1, O_2, O_3, O_4, O_5] = [1, 1, 0, 1, 1]$.

Emission probabilities: $e_0(O = 1) = 0.009$, $e_0(O = 0) = 0.991$, $e_1(O = 1) = 0.985$, $e_1(O = 0) = 0.015$.

t	O_t	$\tilde{\alpha}^{(0)}$	$\tilde{\alpha}^{(1)}$	$\alpha^{(0)}$ (normalised)	$\alpha^{(1)}$ (normalised)	c_t
1	1	—	—	0.0135	**0.9865**	0.3994
2	1	0.0323	0.9677	0.0003	**0.9997**	0.9535
3	0	0.0203	0.9797	**0.5777**	0.4223	0.0348
4	1	0.5457	0.4543	0.0109	**0.9891**	0.4524
5	1	0.0299	0.9701	0.0003	**0.9997**	0.9559

Reading the Forward Table

- $t = 1$: We observe $O_1 = 1$. Since $e_1(1) = 0.985 \gg e_0(1) = 0.009$, the 21CS observation strongly implies $M_1 = 1$: $\hat{P}(M_1 = 1) = 0.9865$.
- $t = 2$: Observing $O_2 = 1$ again. The predicted state before updating: $\tilde{\alpha}^{(0)} = (0.0135)(0.93) + (0.9865)(0.02) = 0.0323$. After the update with $O = 1$: $P(M_2 = 1)$ rises even higher to 0.9997.
- $t = 3$: $O_3 = 0$! No 21CS observed. The update weight for $M = 1$ is now $e_1(0) = p_S = 0.015$ (tiny), so $P(M_3 = 1)$ collapses to 0.4223. The classroom could have slipped out of the productive regime.
- $t = 4$: $O_4 = 1$ again. The state immediately recovers to $P(M_4 = 1) = 0.9891$. This is the near-permanent regime in action: one negative observation cannot permanently kill a productive state when p_S is small.

3.6 Prior Choices Explained

Prior	Mean	Reasoning
$\pi_0 \sim \text{Beta}(2, 3)$	0.40	Slight prior that episodes start unproductively. Most Indian classroom episodes (Behaviourist-dominant) begin with routine instruction.
$p_L \sim \text{Beta}(1.5, 20)$	0.07	Small learning rate expected. Consistent with near-permanent regime hypothesis: productive states are rare achievements, not the default.
$p_F \sim \text{Beta}(1, 50)$	0.02	Forgetting even rarer than learning. Once a productive regime is established, it tends to persist.

3.7 Key Result: Regime Persistence

$$\hat{p}_L = 0.073 \ \ [0.067,\ 0.080], \qquad \hat{p}_F = 0.021 \ \ [0.012,\ 0.033] \tag{26}$$

Average lines per transition: $1/\hat{p}_L \approx 14$. A 384-line episode sees roughly $384 \times 0.073 \approx 28$ transitions total — but most are micro-fluctuations, not sustained regime changes.

3.8 Code

```
EPS       = 1e-7
PS_FIXED = 0.015    # from IO-BKT posterior (fixes label
    switching)
PG_FIXED = 0.009    # from IO-BKT posterior

def pst_forward_single(obs_e, mask_e, pi0, p_L, p_F):
    """
```

```
    obs_e  : shape (Tmax,) -- 21CS observations (0/1), padded
    mask_e : shape (Tmax,) -- 1 for real lines, 0 for padding
    pi0    : scalar        -- P(M_1=1)
    p_L    : scalar        -- learning rate (0->1 per line)
    p_F    : scalar        -- forgetting rate (1->0 per line)
    """
    pS = PS_FIXED
    pG = PG_FIXED

    # Initialisation at t=0
    o0 = obs_e[0]
    # Emission: P(O_1 | M_1=0) and P(O_1 | M_1=1)
    e0_0 = jnp.where(o0 == 1., pG,      1. - pG)
    e1_0 = jnp.where(o0 == 1., 1. - pS, pS)
    # alpha_1^(0) and alpha_1^(1), unnormalised
    a0 = (1. - pi0) * e0_0
    a1 = pi0        * e1_0
    # Normalise and log the normalisation constant
    c0 = a0 + a1 + EPS
    a0 = a0 / c0;  a1 = a1 / c0
    ll0 = jnp.log(c0) * mask_e[0]  # 0 if padding line

    # Recursion: t=1, 2, ..., Tmax-1
    def step(carry, t):
        a0, a1, ll = carry
        o_t = obs_e[t]
        m_t = mask_e[t]   # padding guard

        # PREDICT: marginalise over M_{t-1}
        pred0 = a0 * (1. - p_L) + a1 * p_F        # Eq.(12)
        pred1 = a0 * p_L        + a1 * (1. - p_F)  # Eq.(13)

        # Emission probabilities for current observation
        e0 = jnp.where(o_t == 1., pG,      1. - pG)
        e1 = jnp.where(o_t == 1., 1. - pS, pS)

        # UPDATE: weight by emission
        b0 = pred0 * e0
        b1 = pred1 * e1
        c  = b0 + b1 + EPS

        # Accumulate log-likelihood (zero for padding)
        return (b0/c, b1/c, ll + jnp.log(c) * m_t), None

    # lax.scan: JIT-compiled loop, runs efficiently on GPU
    (_, _, ll_final), _ = lax.scan(
        step, (a0, a1, ll0), jnp.arange(1, Tmax)
    )
    return ll_final
```

Listing 3: PST: Forward algorithm for one episode

```python
# Vectorise forward pass: apply to all E episodes at once
pst_forward_all = vmap(pst_forward_single,
                       in_axes=(0, 0, None, None, None))

def pst_model(obs, mask):
    """
    Only 3 parameters are estimated -- content-free by design.
    """
    # Beta(2,3): mean=0.40, slight prior for unproductive start
    pi0 = numpyro.sample("pi0", dist.Beta(2.0,  3.0))
    # Beta(1.5,20): mean=0.07, small learning rate expected
    p_L = numpyro.sample("p_L", dist.Beta(1.5, 20.0))
    # Beta(1,50): mean=0.02, forgetting even rarer
    p_F = numpyro.sample("p_F", dist.Beta(1.0, 50.0))

    # Forward pass on all episodes simultaneously
    ll = pst_forward_all(obs, mask, pi0, p_L, p_F)
    # numpyro.factor adds log-prob directly to the model
    numpyro.factor("seq_loglik", jnp.sum(ll))

# Run NUTS MCMC
kernel_pst = NUTS(pst_model, target_accept_prob=0.90)
mcmc_pst   = MCMC(kernel_pst, num_warmup=500,
                  num_samples=1000, num_chains=2)
mcmc_pst.run(jax.random.PRNGKey(42), obs_jax, mask_jax)
```

Listing 4: PST: Model and MCMC

3.8.1 Two Technical Details

`jax.lax.scan` A standard Python `for` loop would unroll to $T_{\max}$ Python-level operations, each launching a separate GPU kernel — very slow. `lax.scan` compiles the entire loop into a single XLA kernel, running the forward pass on GPU in milliseconds.

The `mask` trick All episodes are zero-padded to a uniform length $T_{\max}$. The mask ensures padding lines contribute $\log(c_t) \times 0 = 0$ to the log-likelihood. Without masking, the algorithm would treat padding zeros as real "no-21CS" observations, artificially inflating the estimated p_F.

3.9 Why PST's AUC is Misleading

PST achieves AUC = 0.928, higher than IO-BKT's 0.873. But this is not a genuine win.

PST AUC is Autocorrelation, Not Insight

PST's one-step-ahead prediction of O_t is:

$$\hat{P}(O_t = 1 \mid O_1, \ldots, O_{t-1}) \approx O_{t-1}$$

The correlation $\text{corr}(P_t, O_{t-1}) = 0.998$ confirms this. PST is essentially saying "the next observation looks like the current one," which is a tautology when regimes barely change. There is no pedagogical insight: IO-BKT's 0.873, achieved with *content features alone* (not the observation history), is the honest comparison.

4 Model 3: IO-BKT — Input-Output Bayesian Knowledge Tracing

4.1 Core Question

IO-BKT Answers

Which EP/KD/CP codes are most strongly associated with transitions into and out of the productive discourse regime at each utterance? The transition coefficients $\boldsymbol{\beta}_{\text{on}}$ and $\boldsymbol{\beta}_{\text{off}}$ quantify these associations from observational data; causal interpretation requires replication with experimental or quasi-experimental designs.

4.2 The Key Innovation: Content-Driven Transitions

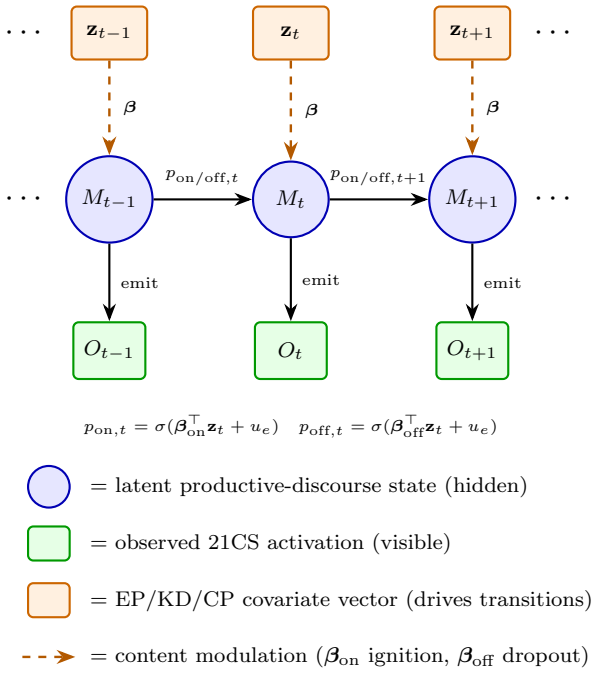


Figure 2: IO-BKT graphical model. At each line t, the observed EP/KD/CP covariate vector $\mathbf{z}_t$ modulates both the 0→1 transition probability (ignition, via $\boldsymbol{\beta}_{\text{on}}$) and the 1→0 transition probability (dropout, via $\boldsymbol{\beta}_{\text{off}}$), making transitions content-driven rather than fixed. The latent productive-discourse state M_t is inferred via the forward algorithm; 21CS activation O_t is the observed emission. Compare PST (Fig. 1), which has no $\mathbf{z}_t$ row and uses constant transition probabilities.

PST uses a constant transition probability: every line has the same p_L (chance of $0 \to 1$)

regardless of whether the teacher is asking "What is the formula?" or "Now apply this to design a bridge." This is clearly wrong.

IO-BKT replaces constant transitions with **sigmoid-linear functions of $\mathbf{z}_t$**:

$$\boxed{P(M_t = 1 \mid M_{t-1} = 0,\ \mathbf{z}_t) = \sigma\big(\boldsymbol{\beta}_{\text{on}}^\top \mathbf{z}_t + u_e\big)} \tag{27}$$

$$\boxed{P(M_t = 0 \mid M_{t-1} = 1,\ \mathbf{z}_t) = \sigma\big(\boldsymbol{\beta}_{\text{off}}^\top \mathbf{z}_t + u_e\big)} \tag{28}$$

The Two Coefficient Vectors Have Distinct Roles

- $\boldsymbol{\beta}_{\text{on}}$ (**ignition coefficients**): $\beta_{\text{on},f} > 0$ means feature f *promotes* entry into the productive state from unproductive. $\beta_{\text{on},f} < 0$ means feature f *suppresses* ignition.
- $\boldsymbol{\beta}_{\text{off}}$ (**dropout coefficients**): $\beta_{\text{off},f} > 0$ means feature f *promotes dropout* from productive to unproductive. $\beta_{\text{off},f} < 0$ means feature f *prevents dropout* (sustains the productive state).

A feature can have *opposite* signs in $\boldsymbol{\beta}_{\text{on}}$ and $\boldsymbol{\beta}_{\text{off}}$ — for example, Applying has $\beta_{\text{on}} = +3.41$ (ignites) *and* $\beta_{\text{off}} = -3.40$ (sustains). A simple regression or correlation cannot separate these roles.

4.3 Full Model Specification

4.3.1 Priors

$$\boldsymbol{\beta}_{\text{on}},\ \boldsymbol{\beta}_{\text{off}} \sim \mathcal{N}(\mathbf{0},\ \mathbf{I}_{14}) \tag{29}$$

$$\pi_0 \sim \text{Beta}(2,\ 3) \tag{30}$$

$$p_S \sim \text{Beta}(2,\ 30)\ \ [\text{mean} \approx 0.06] \tag{31}$$

$$p_G \sim \text{Beta}(1.5,\ 30)\ \ [\text{mean} \approx 0.05] \tag{32}$$

$$\sigma_{\text{cell}} \sim \text{Half-Normal}(1) \tag{33}$$

$$u_e \sim \mathcal{N}(0,\ \sigma_{\text{cell}}^2) \tag{34}$$

Unlike PST, IO-BKT *estimates* p_S and p_G. The content features break label-switching symmetry: by the time MCMC runs, it knows Applying (+3.41) is associated with ignition, so the "productive" label attaches to the state where high-CP lines drive entry — there is no ambiguity.

4.3.2 Emission Model

Unchanged from PST (Eqs. 13–16), but now p_S and p_G are *estimated*:

$$P(O_t = 1 \mid M_t = 1) = 1 - p_S, \qquad P(O_t = 1 \mid M_t = 0) = p_G \tag{35}$$

The posterior means are $\hat{p}_S = 0.015$, $\hat{p}_G = 0.009$ — confirming PST's fixed values were appropriate.

4.3.3 Forward Algorithm with Time-Varying Transitions

The forward algorithm is identical in structure to PST, but now p_L and p_F are replaced by line-specific values $p_{\text{on},t}$ and $p_{\text{off},t}$:

Predict step:

$$\tilde{\alpha}_t^{(0)} = \alpha_{t-1}^{(0)}\,(1 - p_{\text{on},t}) + \alpha_{t-1}^{(1)}\,p_{\text{off},t} \tag{36}$$

$$\tilde{\alpha}_t^{(1)} = \alpha_{t-1}^{(0)}\,p_{\text{on},t} + \alpha_{t-1}^{(1)}\,(1 - p_{\text{off},t}) \tag{37}$$

Update step: same as PST (weight by emission). The key difference: every line has its *own* $p_{\text{on},t}$ and $p_{\text{off},t}$, computed from $\mathbf{z}_t$ via sigmoid.

4.4 Worked Numerical Example: Transition Probabilities

Using the paper's posterior means ($\boldsymbol{\beta}_{\text{on}}$ and $\boldsymbol{\beta}_{\text{off}}$ from Table II, intercept $u_e = 0$):

Line A — Behaviourism + Remembering: $\mathbf{z}_{\text{A}} = [\underbrace{1,0,0}_{\text{EP}},\underbrace{0,0,0,0}_{\text{KD}},\underbrace{1,0,0,0,0,0}_{\text{CP}},1]$

$$\boldsymbol{\beta}_{\text{on}}^{\top}\mathbf{z}_{\text{A}} = (-1.50)_{\text{Beh}} + (-1.15)_{\text{Rem}} + (-1.78)_{\text{int}} = -4.43$$

$$p_{\text{on,A}} = \sigma(-4.43) = \mathbf{0.012} \;\; (1.2\%\text{ ignition chance})$$

Line B — Constructivism + Applying: $\mathbf{z}_{\text{B}} = [\underbrace{0,0,1}_{\text{EP}},\underbrace{0,0,0,0}_{\text{KD}},\underbrace{0,0,1,0,0,0}_{\text{CP}},1]$

$$\boldsymbol{\beta}_{\text{on}}^{\top}\mathbf{z}_{\text{B}} = (+1.25)_{\text{Con}} + (+3.41)_{\text{App}} + (-1.78)_{\text{int}} = +2.88$$

$$p_{\text{on,B}} = \sigma(+2.88) = \mathbf{0.947} \;\; (94.7\%\text{ ignition chance})$$

What This Means for a Teacher

If a teacher is giving a Behaviourist recall drill (Line A type), each line has only a 1.2% chance of igniting a productive discourse regime. But if the teacher shifts to a Constructivist application task (Line B type), the probability jumps to 94.7%. The IO-BKT coefficients operationalise precisely *which* moves matter and by how much.

4.5 Interpreting Table II: The Posterior Coefficients

Ignition ($\boldsymbol{\beta}_{\text{on}}$)			**Dropout prevention / promotion ($\boldsymbol{\beta}_{\text{off}}$)**		
Feature	**Coeff.**	$\lvert\mu\rvert > 2\text{SD}$	**Feature**	**Coeff.**	$\lvert\mu\rvert > 2\text{SD}$
Applying	+3.41	⋆	Applying	−3.40	⋆
Constructivism	+1.25		Constructivism	−2.47	⋆
Behaviourism	−1.50	⋆	Cognitivism	−1.31	⋆
Remembering	−1.15	⋆	Remembering	+1.23	⋆
Procedural	−0.89	⋆	Understanding	−0.80	⋆
Intercept	−1.78	⋆	Intercept	+0.44	

Reading each coefficient:

Applying, $\beta_{\text{on}} = +3.41^{\star}$ and $\beta_{\text{off}} = -3.40^{\star}$
The dominant lever in both directions. Applying-level cognitive demand (Bloom's level 3: "use knowledge in a new situation") both *ignites* the productive state and *sustains* it once established. It is the most impactful feature in the model.

Constructivism, $\beta_{\text{off}} = -2.47^{\star}$
Once the classroom is productive, Constructivist moves (Vygotsky zone-of-proximal-development scaffolding, collaborative discovery) are the strongest *prevention of dropout*. Note: $\beta_{\text{on}} = +1.25$ for Constructivism is *not* credibly non-zero (< 2SD), meaning Constructivism sustains but does not reliably *ignite* (likely due to its extreme rarity in the corpus: $< 5\%$ of lines).

Behaviourism, $\beta_{\text{on}} = -1.50^{\star}$
Operant-conditioning-style instruction (drill, rote recall, reward/punish) actively *suppresses* entry into the productive regime.

Remembering, $\beta_{\text{on}} = -1.15^{\star}$ and $\beta_{\text{off}} = +1.23^{\star}$
Pure recall tasks both suppress ignition *and* are the most reliable dropout trigger. If the classroom is productive and the teacher reverts to "remember this definition," the productive state is at serious risk.

Intercept, $\beta_{\text{on}} = -1.78^{\star}$
The baseline ignition probability (when all features are zero) is $\sigma(-1.78) = 0.144$. **Classrooms default toward unproductive discourse**: productivity requires active effort.

4.6 Code

```
def forward_single(obs_e, mask_e, pon_e, poff_e, pi0, pS, pG):
    """
    pon_e  : shape (Tmax,) -- P(M_t=1|M_{t-1}=0, z_t), per line
    poff_e : shape (Tmax,) -- P(M_t=0|M_{t-1}=1, z_t), per line
    These vary each line -- the key difference from PST!
    """
    # Initialise
    o0   = obs_e[0]
    e0_0 = jnp.where(o0==1., pG, 1.-pG)      # P(O1|M1=0)
    e1_0 = jnp.where(o0==1., 1.-pS, pS)      # P(O1|M1=1)
    a0   = (1.-pi0) * e0_0
    a1   = pi0      * e1_0
    c0   = a0 + a1 + EPS
    a0  /= c0;  a1 /= c0
    ll0  = jnp.log(c0) * mask_e[0]

    def step(carry, t):
        a0, a1, ll = carry
        o_t    = obs_e[t]
        m_t    = mask_e[t]
```

```
        pon_t  = pon_e[t]    # content-driven -- changes each
            line
        poff_t = poff_e[t]   # content-driven -- changes each
            line

        # PREDICT: Eqs. (16)-(17)
        pred0 = a0*(1.-pon_t) + a1*poff_t
        pred1 = a0*pon_t      + a1*(1.-poff_t)

        # UPDATE: weight by emission
        e0 = jnp.where(o_t==1., pG, 1.-pG)
        e1 = jnp.where(o_t==1., 1.-pS, pS)
        b0 = pred0*e0;  b1 = pred1*e1
        c  = b0 + b1 + EPS

        return (b0/c, b1/c, ll + jnp.log(c)*m_t), None

    (_, _, ll_final), _ = lax.scan(
        step, (a0, a1, ll0), jnp.arange(1, Tmax)
    )
    return ll_final

# Vectorise: run forward_single on all E episodes in parallel
forward_all = vmap(forward_single,
                   in_axes=(0, 0, 0, 0, None, None, None))
```

Listing 5: IO-BKT: Forward algorithm with time-varying transitions

```
def iobkt_model(Z, obs, mask, cell_idx, n_cells, Tmax, P):
    """
    Z       : shape (E, Tmax, P) -- feature vectors, all
       episodes
    obs     : shape (E, Tmax)    -- 21CS observations
    mask    : shape (E, Tmax)    -- padding mask
    """
    E = Z.shape[0]

    # Global coefficient vectors (14D)
    # b_on[f]:  how feature f drives productive ignition
    # b_off[f]: how feature f drives productive dropout
    b_on  = numpyro.sample("b_on",
                dist.Normal(jnp.zeros(P), jnp.ones(P)))
    b_off = numpyro.sample("b_off",
                dist.Normal(jnp.zeros(P), jnp.ones(P)))

    #  Subject x Grade random effects
    sigma_cell = numpyro.sample("sigma_cell",
                                dist.HalfNormal(1.0))
    u_cell = numpyro.sample("u_cell",
                 dist.Normal(jnp.zeros(n_cells),
```

```
                                sigma_cell * jnp.ones(n_cells)))

    # HMM parameters
    pi0 = numpyro.sample("pi0", dist.Beta(2.0,  3.0))
    pS  = numpyro.sample("pS",  dist.Beta(2.0, 30.0)) # mean ~0.06
    pG  = numpyro.sample("pG",  dist.Beta(1.5, 30.0)) # mean ~0.05

    # Compute time-varying transition probabilities
    u_e = u_cell[cell_idx]            # shape: (E,)
    # eta_on[e,t]  = sum_p (Z[e,t,p] * b_on[p]) + u_e[e]
    # "etp,p->et" means: sum over last axis (features p)
    eta_on  = jnp.einsum("etp,p->et", Z, b_on)  + u_e[:, None]
    eta_off = jnp.einsum("etp,p->et", Z, b_off) + u_e[:, None]
    # Squeeze through sigmoid to get probabilities
    p_on  = jax.nn.sigmoid(eta_on)   # shape: (E, Tmax)
    p_off = jax.nn.sigmoid(eta_off)  # shape: (E, Tmax)
    # p_on[e,t]  = P(M_t=1 | M_{t-1}=0, z_{e,t})
    # p_off[e,t] = P(M_t=0 | M_{t-1}=1, z_{e,t})

    # Run forward algorithm over all episodes
    ll_per_episode = forward_all(obs, mask, p_on, p_off,
                                 pi0, pS, pG)
    numpyro.factor("seq_loglik", jnp.sum(ll_per_episode))
```

Listing 6: IO-BKT: NumPyro model with hierarchical effects

4.6.1 The `einsum` Operation Decoded

```
eta_on = jnp.einsum("etp,p->et", Z, b_on) + u_e[:, None]
```

This computes for all episodes e and time steps t simultaneously:

$$\eta_{\text{on},e,t} = \sum_{p=1}^{P} Z_{e,t,p} \cdot \beta_{\text{on},p} + u_e \;=\; \boldsymbol{\beta}_{\text{on}}^{\top} \mathbf{z}_{e,t} + u_e$$

The index string `"etp,p->et"` means: *sum over dimension p (features), keeping dimensions e (episodes) and t (time steps).* This replaces a nested Python loop with a single GPU matrix operation. Without `einsum`, you would write:

```
# Equivalent but slow:
for e in range(E):
    for t in range(Tmax):
        eta_on[e,t] = sum(Z[e,t,:] * b_on) + u_e[e]
```

4.6.2 MCMC Configuration

```
kernel = NUTS(
```

```
    iobkt_model,
    target_accept_prob=0.95,    # higher than BLFM's 0.90
    max_tree_depth=10,          # allow longer HMC trajectories
)
mcmc = MCMC(
    kernel,
    num_warmup=500,
    num_samples=1000,
    num_chains=2,
    chain_method='sequential', # avoid GPU memory doubling
    progress_bar=True,
)
```

Listing 7: IO-BKT: NUTS with tighter tolerances for complex geometry

Why `target_accept_prob=0.95`? IO-BKT's posterior has a more complex, curved geometry than BLFM's, because the 28 coefficients ($\boldsymbol{\beta}_{\text{on}}$ and $\boldsymbol{\beta}_{\text{off}}$) interact with the forward algorithm non-linearly. A higher target acceptance forces NUTS to take smaller, more careful steps (via a smaller step size), reducing the number of divergent transitions at the cost of slower exploration. This is the standard recommendation when you see many divergences with the default 0.80 or 0.90.

4.7 Label Switching: Detection and Correction

```
pS_mean = float(np.array(samples["pS"]).mean())
if pS_mean > 0.5:
    # The model learned the states in REVERSE order.
    # "State 1" actually behaves like the unproductive state.
    print("Label switching detected -- flipping states")

    # Flip all coefficient signs: if b_on drove 0->1 transition
    # in the wrong direction, negating it flips the roles.
    samples["b_on"]  = -samples["b_on"]
    samples["b_off"] = -samples["b_off"]

    # Correct emission parameters.
    # Original "state 1" had slip-like pS = 1 - pG_correct.
    # After flipping:  new pS = 1 - old pG,  new pG = 1 - old
       pS
    pS_old = samples["pS"].copy()
    pG_old = samples["pG"].copy()
    samples["pS"] = 1.0 - pG_old
    samples["pG"] = 1.0 - pS_old
```

Listing 8: Label switching check and correction after MCMC

Why does this happen? If MCMC's random initialisation starts near the "wrong" mode (where state 0 is actually more productive than state 1), it may converge to the label-swapped solution. The diagnostic: $\hat{p}_S > 0.5$ would mean the "productive" state has a 50%+ chance of producing no 21CS — absurd. Negating the coefficients and swapping

emission parameters corrects this.

4.8 The Full-Length Normalisation (Tmax Extension)

The training used $T_{\max} = 600$ (92nd percentile). Some episodes (e.g. Episode 37 with 921 lines) were truncated. The normalisation extension re-runs *only the forward pass* with $T_{\max} = 950$, keeping all MCMC-estimated parameters unchanged.

```
# b_on_m, b_off_m etc. are the posterior MEANS (fixed, from MCMC)
# We just extend the input arrays and re-apply the forward pass.

for i in range(E):
    T_new = int(mask_full[i].sum())   # now using all lines
    u_e   = float(u_cell_m[cell_idx[i]])

    # Time-varying transitions for ALL T_new lines
    # (same formula as training, just more timesteps)
    pon_e  = sigmoid(Z_full[i, :T_new] @ b_on_m  + u_e)
    poff_e = sigmoid(Z_full[i, :T_new] @ b_off_m + u_e)

    # Run the same forward algorithm over extended sequence
    probs = forward_pass(obs_full[i, :T_new], pon_e, poff_e, ...)
```

Listing 9: Forward pass extension without re-running MCMC

This is valid because the IO-BKT coefficients describe how EP/KD/CP features drive transitions *in general*, they are not specific to the first 600 lines. Applying them to lines 601–950 is legitimate extrapolation, not re-fitting.

5 Results, Comparison, and the Three-Stage Pipeline

5.1 Model Performance Comparison

5.2 Baselines

The three baselines span the design space and each serves a specific evidentiary role; they were not selected for convenience but to answer three distinct questions about what the proposed framework adds over existing tools.

Frequentist BKT (pyBKT) is the field's default sequential KT method. We run BKT aggregate (one KC across all lines) and BKT per-KC (dominant EP as the KC label), each with 5 random initialisations. Its role is diagnostic: if standard BKT works, the problem is already solved. That both variants collapse to AUC = 0.500 is itself a result – it establishes that near-permanent regimes are precisely the failure mode classical KT is not designed for, because the filtered posterior assigns near-constant within-episode probabilities when p_L is small, losing all within-episode discrimination.

DKT-LSTM (content-only): 1-layer LSTM (hidden = 64, dropout = 0.2), input , Adam (lr= 10^{-3}, wd= 10^{-4}), 30 epochs. It uses identical inputs to IO-BKT but replaces the interpretable HMM with a recurrent black box. DKT is evaluated by leave-one-episode-out CV (LOO, 48 folds), giving it a *stricter* evaluation than the in-sample Bayesian models. This asymmetry is principled: Bayesian priors act as regularisation, making in-sample posterior predictive distributions analogous to cross-validated frequentist estimates. That IO-BKT wins by +0.201 AUC *despite* DKT's evaluation advantage establishes that principled Bayesian structure outperforms neural flexibility at this corpus size. PSIS-LOO comparison across all models is not reported because PST and IO-BKT sequence log-likelihoods (∼390 per-line contributions) are incommensurable with BLFM's single Beta observation per episode.

BLFM (§2) is the exchangeable Bayesian baseline from our own framework. Its inclusion provides a within-framework ablation: the AUC gap to IO-BKT (+0.101) isolates precisely what temporal structure with content-driven transitions contributes over the episode-level sufficient statistic.

Method	Type	Acc	AUC	F1
Group A: Content-only (fair comparison to DKT)				
IO-BKT (Bayesian)	Seq., in-sample	0.878	**0.873**	0.875
BLFM (Bayesian)	Exch., in-sample	0.700	0.772	0.669
DKT-LSTM	Seq., LOO	0.623	0.672	0.575
Group B: Uses observation history (AUC reflects autocorrelation)				
PST (Bayesian)	Seq., in-sample	0.880	0.928*	0.876
BKT agg. (freq.)	Seq., in-sample	0.518	0.500	0.000

* $\text{corr}(P_t, O_{t-1}) = 0.998$: AUC from regime persistence, not content.

5.3 What Each AUC Gap Means

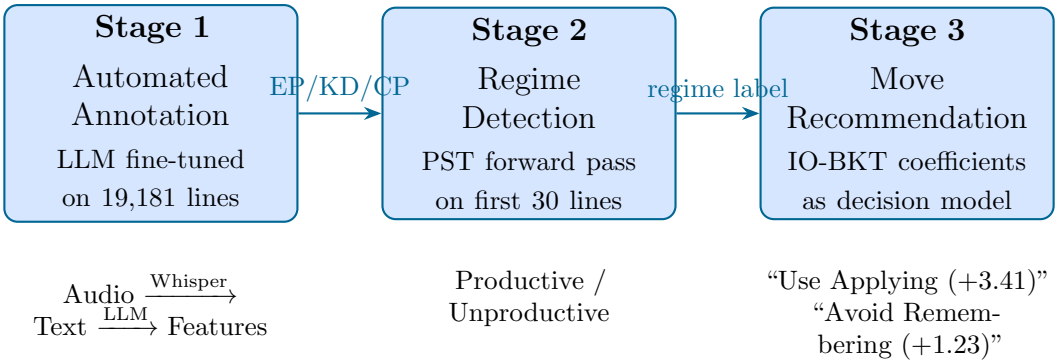
BLFM → IO-BKT: +0.101 Adding temporal structure with content-driven transitions is worth 10 AUC points. The episode-mean sufficient statistic throws away real sequential information.

DKT → IO-BKT: +0.201 Despite IO-BKT being evaluated in-sample while DKT uses stricter LOO-CV, IO-BKT wins. Principled Bayesian priors serve as implicit regularisation, outperforming a data-hungry neural network on a small ($E = 48$ episodes), diverse corpus.

IO-BKT → PST: +0.055 *Not* a genuine improvement. This gap is pure autocorrelation. On data where regimes are near-permanent, predicting "same as last line" trivially achieves high AUC.

5.4 The Three-Stage AI Coaching Pipeline

The paper's models together define a deployable real-time coaching system:



Stage	Details
1. Annotate	Fine-tune an open-source LLM (e.g. Llama 3) on the 19,181 expert-annotated lines to predict EP/KD/CP. Reduces cost from ∼3 person-hours per episode to seconds. Classroom audio via Whisper completes the pipeline.
2. Detect	Run PST's forward algorithm on the first 30 lines of the episode to classify whether the lesson is on a productive trajectory.
3. Recommend	Use IO-BKT's β_{on} and β_{off} as a portable decision model in a teacher-facing app: • Ignite: increase Applying-level demand (+3.41) • Sustain: deploy Constructivism (−2.47 on dropout) • Avoid: Remembering-level drops (+1.23 dropout trigger)

6 Summary: The Three Models Side by Side

Property	BLFM	PST	IO-BKT
Data unit	Episode mean $\bar{\mathbf{x}}_e$	Line sequence $O_{1:T}$	Lines $O_{1:T}$ + features $\mathbf{z}_{1:T}$
Latent state	None (regression)	$M_t \in \{0, 1\}$	$M_t \in \{0, 1\}$
Transition	N/A	Fixed p_L, p_F (content-free)	$\sigma(\boldsymbol{\beta}^\top \mathbf{z}_t)$ per line
Emission	Beta regression	Fixed p_S, p_G	Estimated p_S, p_G
Parameters	$\mathbf{w}$, b, ϕ, σ_{cell}	π_0, p_L, p_F	$\boldsymbol{\beta}_{\text{on}}$, $\boldsymbol{\beta}_{\text{off}}$, π_0, p_S, p_G, σ_{cell}
Key question	Does content mix predict 21CS rate?	How stable are discourse regimes?	Which moves ignite/extinguish productive talk?
Key finding	21CS rate $\propto$ content mix	$\hat{p}_L = 0.073$: regimes near-permanent	Applying: +3.41; Constructivism: −2.47; Remembering: +1.23
Runtime	< 5 min (CPU)	$\sim$15 min (GPU)	$\sim$180 min (GPU)
AUC	0.772	0.928†	**0.873**
Use when	Bulk analysis, no per-line prediction needed	Measuring regime stability, early classification	Identifying causal moves, AI coaching

† Inflated by autocorrelation ($\text{corr}(P_t, O_{t-1}) = 0.998$).

Conclusion

The three models form a deliberate progression:

1. **BLFM** establishes the baseline: yes, content predicts 21CS, and the AUC gap (0.772 vs 0.873) quantifies exactly what is lost by discarding temporal order.
2. **PST** measures the regime dynamics: transitions are rare ($\hat{p}_L = 0.073$, $\hat{p}_F = 0.021$). The classroom spends most of its time in one regime. This empirically *justifies* BLFM's exchangeability assumption — and motivates early-detection tools (classify by line 30; the regime will likely hold).
3. **IO-BKT** provides the actionable layer: *what causes* transitions. Applying-level cognitive demand (+3.41) is the primary ignition driver. Constructivism (−2.47 on

dropout) sustains productive discourse once established. Remembering (+1.23) is the most reliable regime killer. These coefficients are not descriptive statistics — they are a portable, deployable decision model for the next generation of AI teaching coaches.

The near-permanent regime finding is specific to this predominantly Behaviourist, India-dominant corpus and requires replication in Socratic, inquiry-based, or dialogic classrooms before strong deployment claims are made. Subject to that caveat, these three models provide the first principled Bayesian framework for tracking classroom-level productive discourse — the missing upstream layer that student-level knowledge tracing requires.